

LAB - 0

AIM: To study SQL commands, their uses, syntax, and install MySQL for database management experiments.

1. What is SQL and its Use?

SQL is a standard language for accessing and manipulating databases. SQL stands for Structured Query Language. SQL lets you access and manipulate databases.

Uses of SQL are as follows:

- SQL can execute queries against a database
- SQL can retrieve data from a database
- SQL can insert records in a database
- SQL can update records in a database
- SQL can delete records from a database
- SQL can create new databases
- SQL can create new tables in a database
- SQL can create stored procedures in a database
- SQL can create views in a database
- SQL can set permissions on tables, procedures, and views

2. Describe all the different commands of SQL, mention use of it and syntax.

1. SELECT - extracts data from a database

The **SELECT** statement is used to select data from a database.

SELECT Syntax

```
column1, column2, SELECT ...  
FROM table_name;
```

Here, *column1*, *column2*, ... are the **column names** in the table you want to select data from.

The *table_name* represents the **name of the table** you want to select data from.

2. UPDATE - updates data in a database

The **UPDATE** statement is used to update or modify one or more records in a table.

UPDATE Syntax

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

when updating records in a table, notice the WHERE clause in the UPDATE statement. The WHERE clause specifies which record(s) that should be updated. If you omit the WHERE clause, all records in the table will be updated!

- The **WHERE** clause is used to filter records.
The **WHERE** clause is used to extract only those records that fulfill a specific condition.

WHERE Syntax

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

3. DELETE - deletes data from a database

The **DELETE** statement is used to delete existing records in a table.

DELETE Syntax

```
DELETE FROM table_name WHERE condition;
```

4. INSERT INTO - inserts new data into a database

The **INSERT INTO** statement is used to insert new records in a table.

It is possible to write the **INSERT INTO** statement in two ways:

Syntax 1

Specify both the column names and the values to be inserted:

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Syntax 2

If you insert values for ALL the columns of the table, you can omit the column names.

However, the order of the values must be in the same order as the columns in the table:

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

5. CREATE DATABASE - creates a new database

The **CREATE DATABASE** statement is used to create a new SQL database.

Syntax

```
CREATE DATABASE database_name;
```

6. ALTER TABLE - modifies a table

The **ALTER TABLE** statement is used to add, delete, or modify columns in an existing table.

The **ALTER TABLE** statement is also used to add and drop various constraints on an existing table.

Common **ALTER TABLE** operations are:

- Add column - Adds a new column to a table
- Drop column - Deletes a column in a table
- Rename column - Renames a column
- Modify column - Changes the data type, size, or constraints of a column
- Add constraint - Adds a new constraint
- Rename table - Renames a table

ALTER TABLE - ADD COLUMN

To add a column in a table, use the following syntax:

Syntax

ALTER TABLE *table_name*

ADD *column_name datatype*;

ALTER TABLE - DROP COLUMN

To delete a column in a table, use the following syntax (notice that some database systems don't allow deleting a column):

Syntax

ALTER TABLE *table_name*

DROP COLUMN *column_name*;

ALTER TABLE - RENAME COLUMN

To rename a column in a table, use the following syntax:

Syntax

ALTER TABLE *table_name*

RENAME COLUMN *old_name* **to** *new_name*;

ALTER TABLE - MODIFY DATATYPE

To modify the data type, size or constraints of a column in a table, use the following syntax:

Syntax for SQL Server / MS Access:

ALTER TABLE *table_name*

ALTER COLUMN *column_name* *new_datatype* *constraint*;

Syntax for MySQL / Oracle:

ALTER TABLE *table_name*

MODIFY *column_name* *new_datatype* *constraint*;

ALTER TABLE - ADD CONSTRAINT

To add a constraint to an existing table, use the following syntax:

Syntax

ALTER TABLE *table_name*

ADD CONSTRAINT *constraint_name constraint_definition*;

ALTER TABLE - RENAME TABLE

To rename a table, use the following syntax:

Syntax

ALTER TABLE *table_name*

RENAME TO *new_table_name*;

7. CREATE TABLE - creates a new table

The **CREATE TABLE** statement is used to create a new table in a database.

Syntax

CREATE TABLE *table_name* (
 column1 datatype constraint,
 column2 datatype constraint,
 column3 datatype constraint,
 );

The *table_name* parameter specifies the name of the new table.

The *column1*, *column2*, ... parameters specify the names of the columns within the table.

The *datatype* parameter specifies the data type of each column (e.g. varchar, int, date, etc.).

The *constraint* parameter is optional, and specifies rules for data integrity (e.g. primary key, not null, etc.)

8. DROP TABLE - deletes a table

The **DROP TABLE** statement is used to permanently delete an existing table in a database.

Syntax

DROP TABLE *table_name*;

To prevent an error from occur (if the table does not exists), it is a good practice to add the **IF EXISTS** clause:

DROP TABLE IF EXISTS *table_name*;

9. CREATE INDEX - creates an index (search key)

The **CREATE INDEX** statement is used to create indexes on tables in databases, to speed up data retrieval.

The users cannot see the indexes, they are just used to speed up searches/queries.

Types of Indexes: Non-unique and Unique

There are two types of indexes:

- **CREATE INDEX** - Creates a non-unique index (duplicate values are allowed)
- **CREATE UNIQUE INDEX** - Creates a unique index (duplicate values are not allowed)

CREATE INDEX Syntax

CREATE INDEX *index_name*

ON *table_name* (*column1*, *column2*, ...);

CREATE UNIQUE INDEX Syntax

CREATE UNIQUE INDEX *index_name*

ON *table_name* (*column1*, *column2*, ...);

Note: The syntax for creating indexes varies among different databases.

10. DROP DATABASE - deletes a database

The DROP DATABASE statement is used to permanently delete an existing database.

Syntax

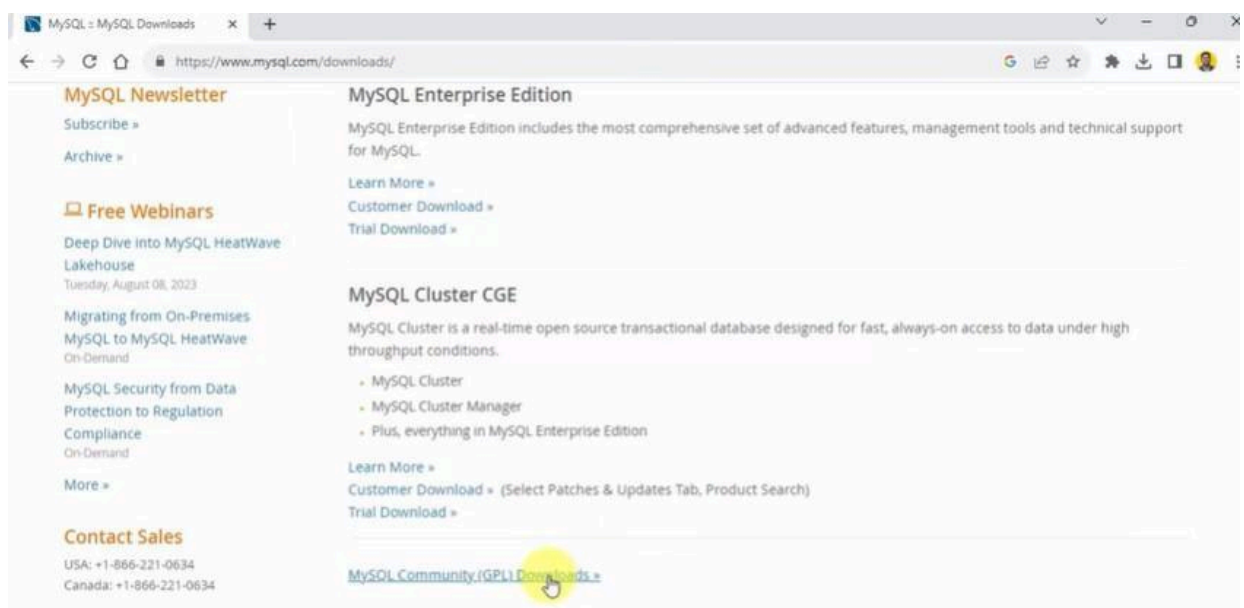
DROP DATABASE *database_name*;

To prevent an error from occurring if the database does not exist, it is a good practice to add the IF EXISTS clause:

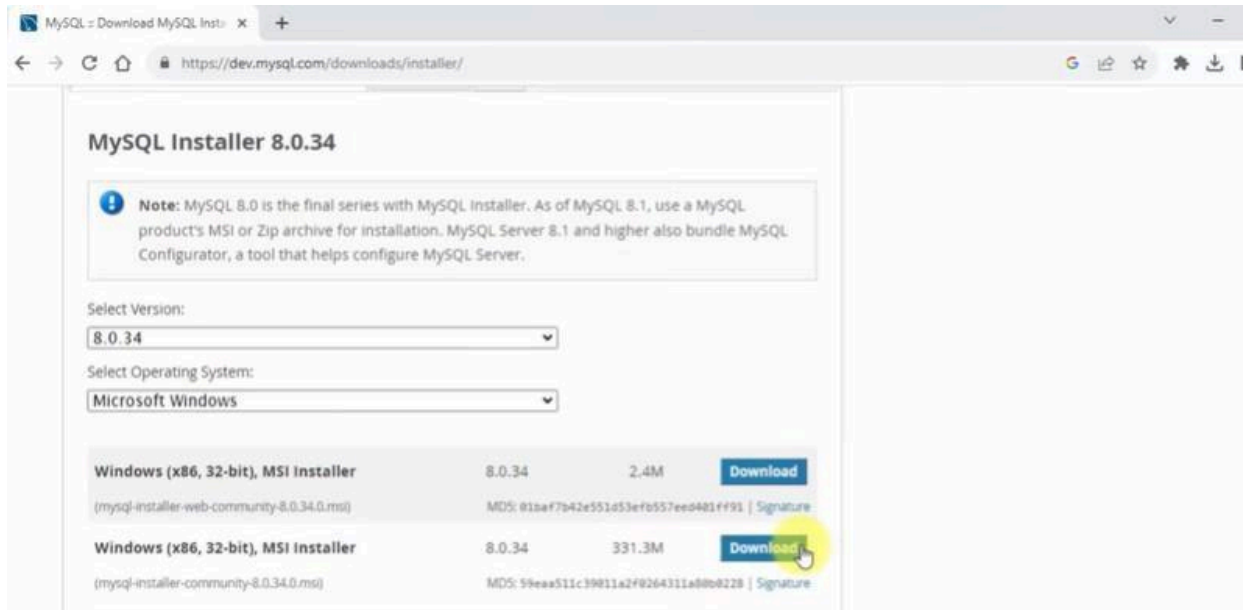
DROP DATABASE IF EXISTS *database_name*;

3. Write a step by step procedure to install MYSQL.

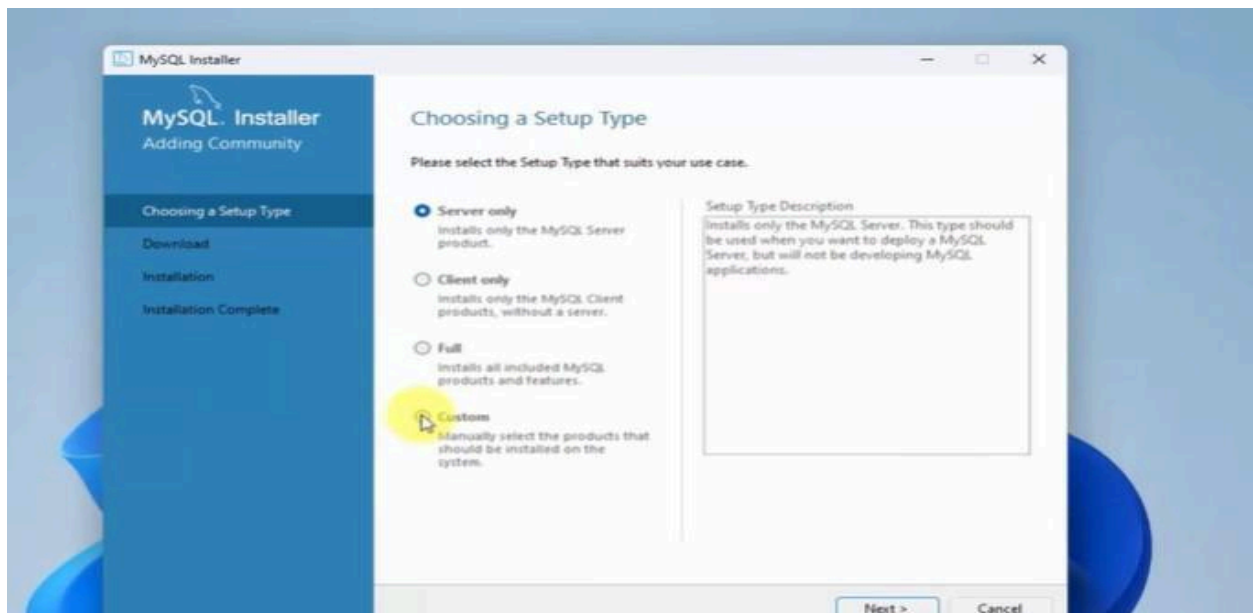
1. Open this link : <https://www.mysql.com/downloads/>
2. Go to Downloads and scroll down



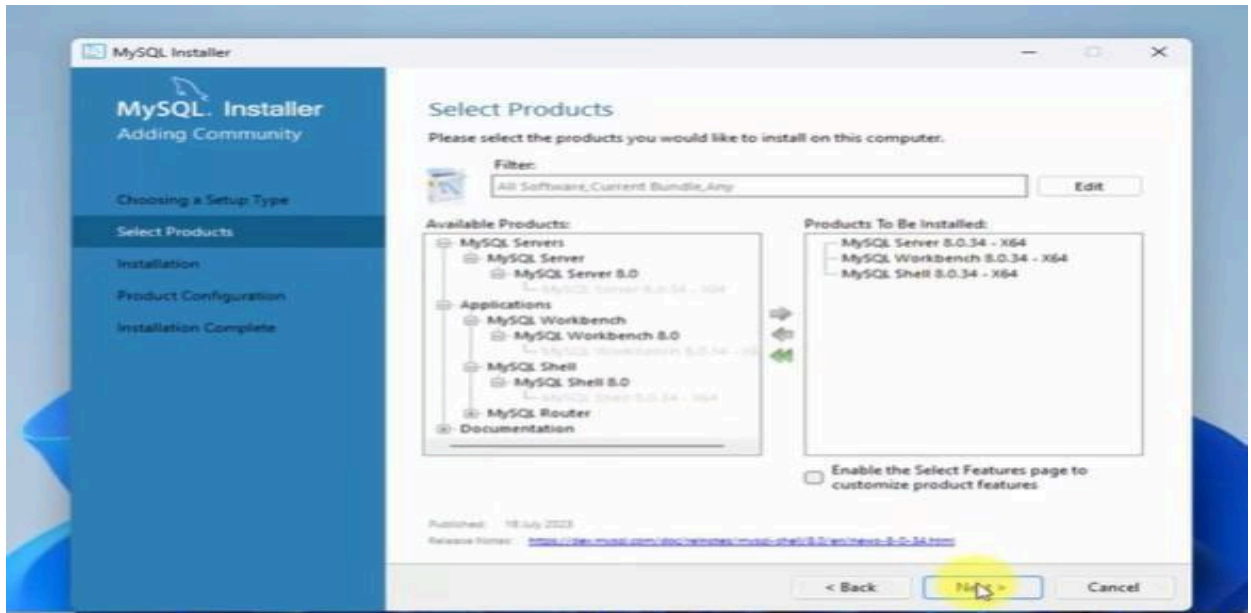
3. Click on **MySQL community downloads**
4. Select installer for windows and select following version for download



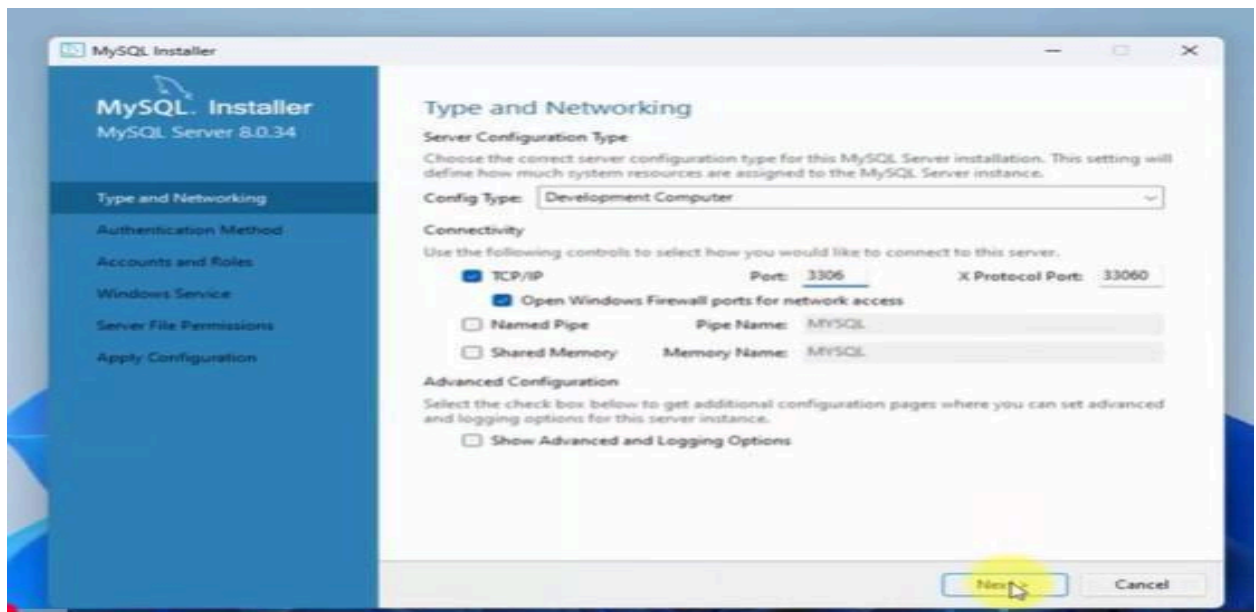
5. Install the downloaded file and then select customs



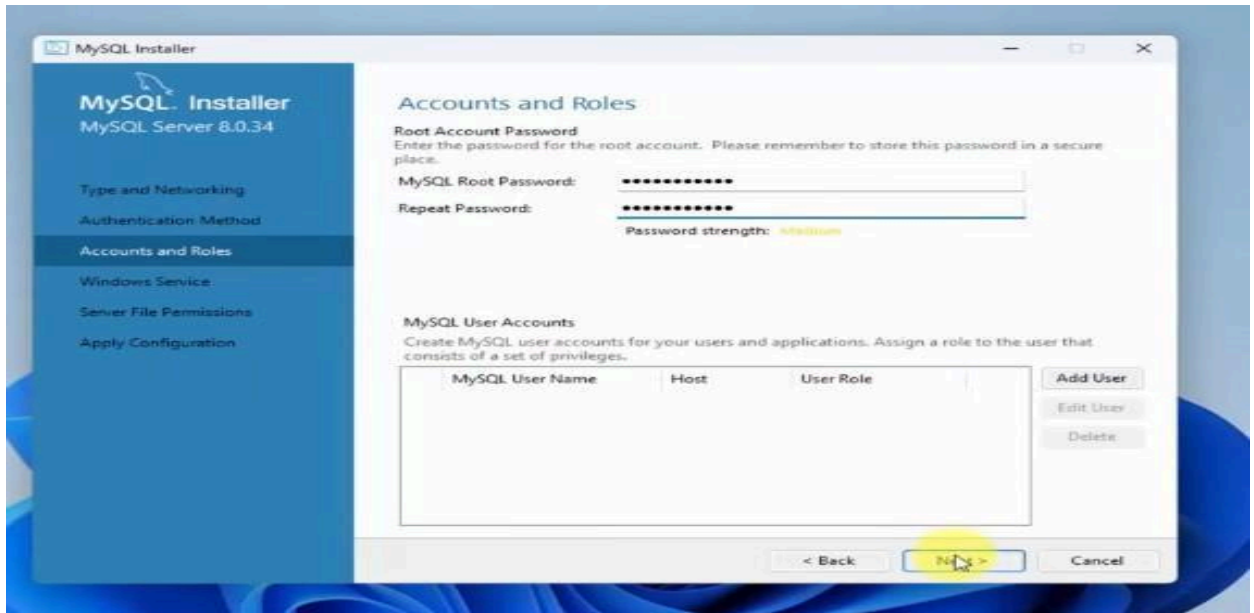
6. Select the required products from the “**Available Products**” drop-down list. Click the **Forward Arrow (→)** to add the selected product to the “**Products To Be Installed**” section. Repeat the process for all the required products, such as **MySQL Server**, **MySQL Workbench**, and **MySQL Shell**. Once all the required products are added, click **Next** to proceed with the installation.



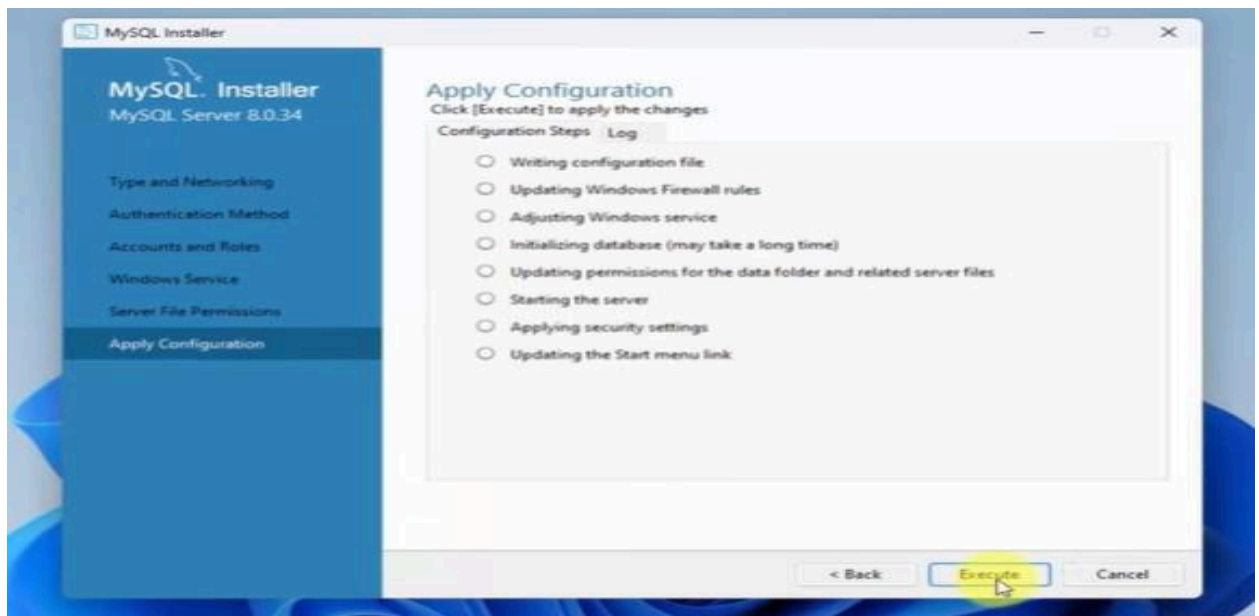
7. Click **Next** to proceed to the **Type and Networking** page. Keep all the **predefined/default values** unchanged, including the configuration type, port numbers, and networking settings. Then, click **Next** to continue.



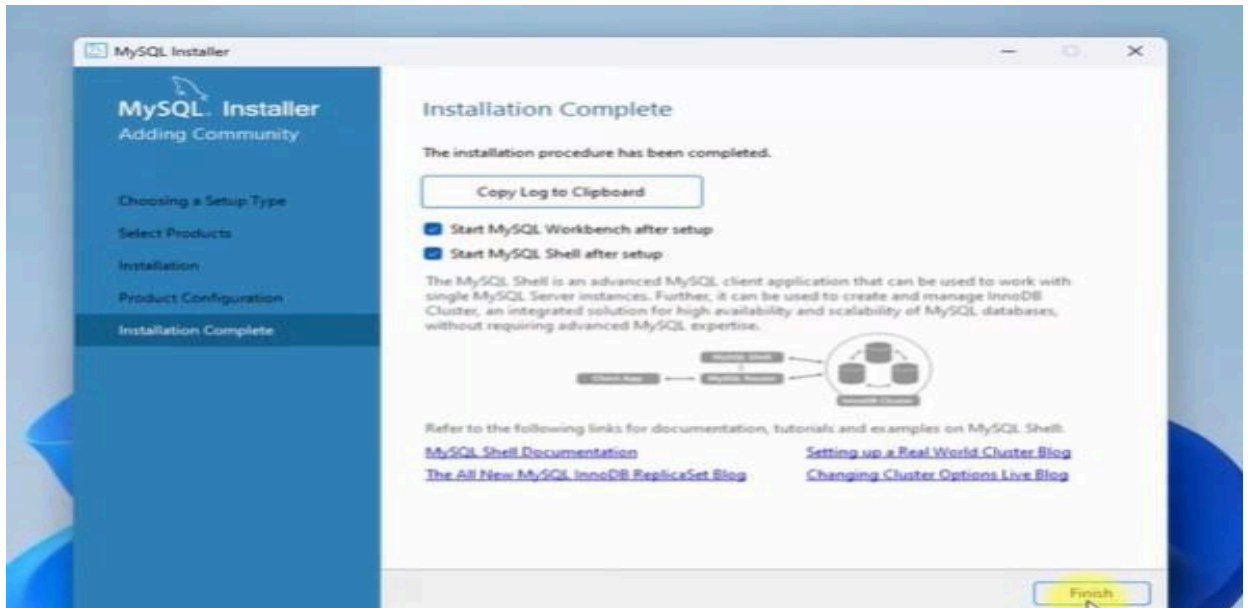
8. Enter a **root account password** in the **MySQL Root Password** field. Make sure to remember this password, as it will be required to access MySQL later. You can leave the **MySQL User Accounts** section unchanged. Then, click **Next** to continue.



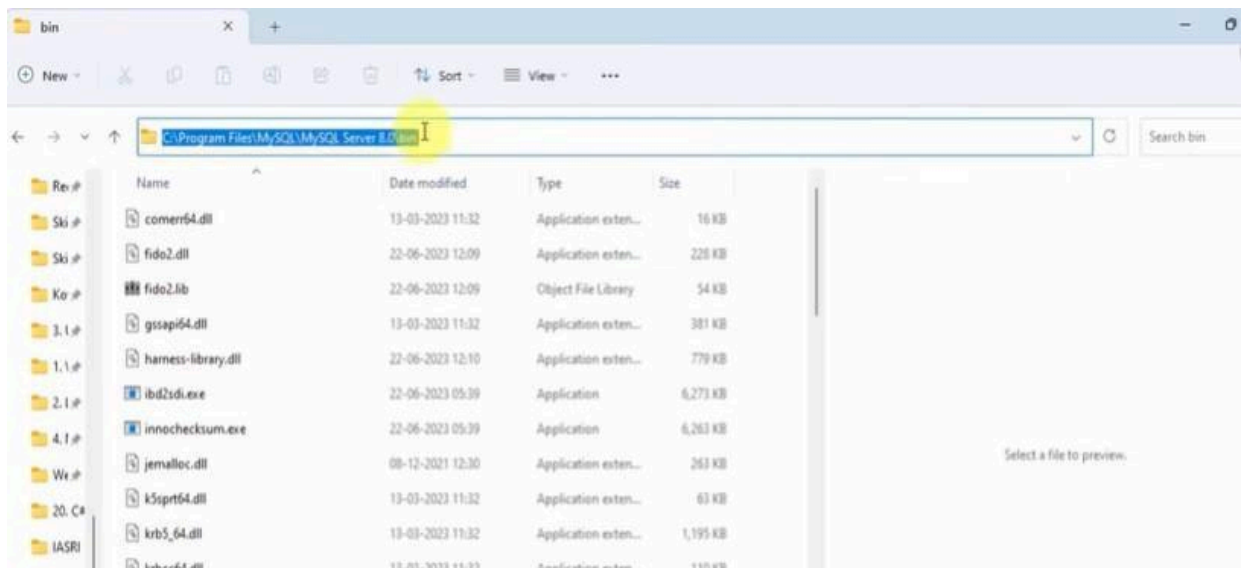
9. Click **Next** on each page without changing the predefined settings until you reach the **Apply Configuration** page. Once you reach this page, click **Execute** to apply the configuration and proceed with the installation.



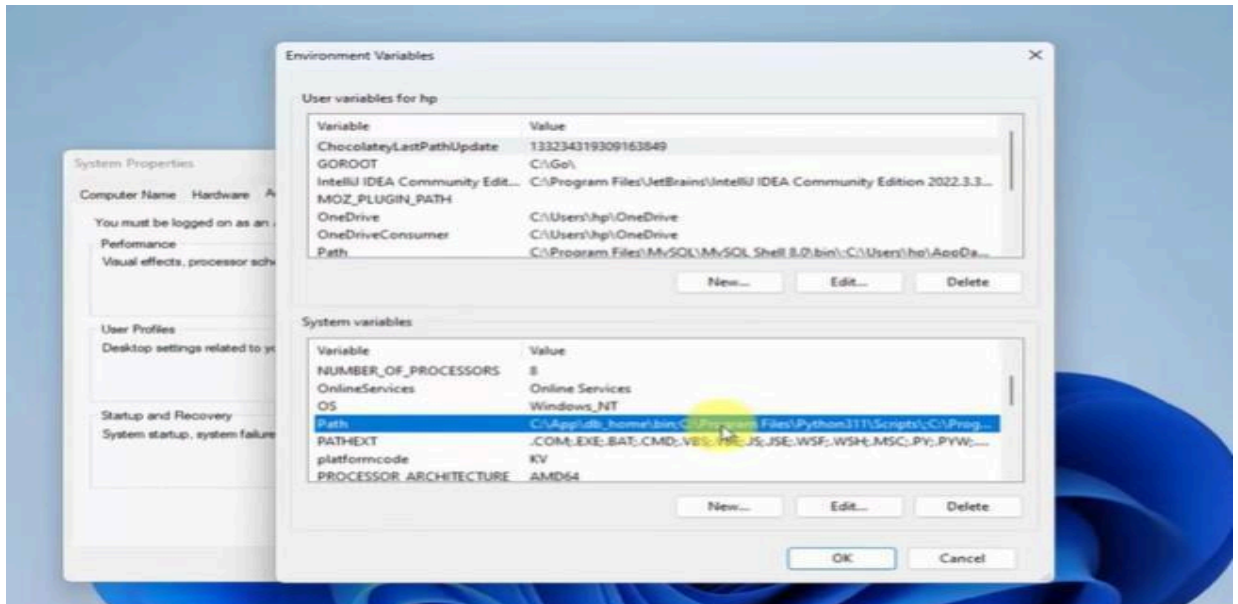
10. Once the configuration is successfully applied, the **Installation Complete** page will appear. Ensure that the required options are selected, then click **Finish** to complete the MySQL installation.



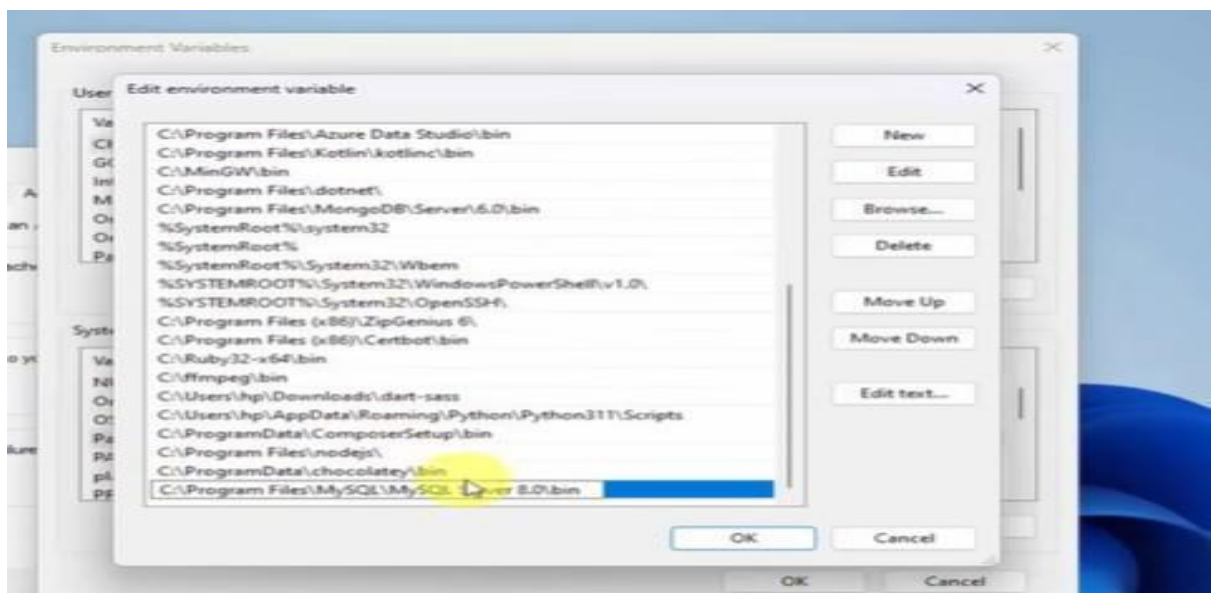
11. To copy the MySQL path, open **This PC → Local Disk (C:) → Program Files → MySQL**. Open the **MySQL Server** folder and then select the **bin** folder. Copy the **path from the address bar**, as this path will be used to configure the MySQL environment variable.



12. Open **System Environment Variables** and click on **Environment Variables**. Under the **System Variables** section, locate and select **Path**. Then, double click on the path.



13. Select **New** and paste the copied path on the new line and click **OK**



14. Open **Command Prompt** and type `mysql --version` and press enter.

